



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

**ESCUELA PROFESIONAL DE INGENIERÍA DE
SISTEMAS**

INFORME DE TRABAJO

Curso: Programación II

Docente: Ing. Breno Solim

Platero Maron, Victor Joseph

**Tacna – Perú
2026**

ÍNDICE

INTRODUCCIÓN	3
MARCO TEÓRICO	3
1. Definición y propósito:	3
2. Características destacadas:	4
3. Aplicaciones y casos de uso:	7
CONCLUSIÓN	9
REFERENCIAS BIBLIOGRÁFICAS	10

INTRODUCCIÓN

En el presente laboratorio se desarrolló una aplicación de escritorio utilizando el lenguaje de programación C# en el entorno Windows Forms. El objetivo principal fue crear un sistema de alineación de fútbol que permita registrar y administrar jugadores convocados para un partido.

La aplicación permite ingresar datos importantes de cada jugador, como nombre, posición, número de camiseta y condición (Titular o Suplente). Además, se implementaron funcionalidades para agregar, modificar, eliminar y mostrar jugadores mediante una interfaz gráfica amigable y fácil de utilizar.

Asimismo, se aplicaron conceptos fundamentales de Programación Orientada a Objetos (POO), utilizando clases, objetos, encapsulamiento, métodos y colecciones (List) para el manejo de la información en memoria, sin necesidad de utilizar una base de datos.

MARCO TEÓRICO

1. Definición y propósito:

La programación en C# dentro del entorno Windows Forms permite desarrollar aplicaciones de escritorio con interfaces gráficas dinámicas e interactivas. Este tipo de aplicaciones facilita la manipulación de información mediante controles visuales y eventos programados.

En el presente laboratorio se desarrolló un sistema de alineación de fútbol, el cual permite gestionar jugadores convocados utilizando operaciones CRUD (Crear, Mostrar, Modificar y Eliminar) en memoria.

La aplicación hace uso de Programación Orientada a Objetos (POO), paradigma que permite organizar el código mediante clases y objetos, facilitando la reutilización y mantenimiento del sistema.

Para el desarrollo del proyecto se utilizaron diversos controles gráficos como:

- Label
- TextBox
- ComboBox
- NumericUpDown
- Button
- DataGridView

Además, se implementó una colección tipo `List<Jugador>` para almacenar temporalmente la información de los jugadores registrados y un archivo .txt para conservar los datos aun cuando la aplicación se cierre.

De esta manera, el sistema permite gestionar de forma sencilla la alineación de un equipo de fútbol mediante una interfaz intuitiva y funcional.

2. Características destacadas:

Interfaz gráfica amigable

El sistema fue diseñado con una interfaz clara, ordenada e intuitiva que permite al usuario registrar y administrar jugadores de manera sencilla mediante controles gráficos de Windows Forms.

Gestión de jugadores

La aplicación permite registrar información importante de cada jugador, como nombre, posición, número de camiseta y condición (Titular o Suplente), facilitando la organización de la alineación del equipo.

Validación de datos

Se implementaron validaciones para asegurar que los campos sean completados correctamente, evitando registros vacíos y errores en el ingreso de información.

Operaciones CRUD

El sistema permite realizar operaciones básicas de administración de datos:

- Agregar jugadores
- Modificar jugadores
- Eliminar jugadores
- Mostrar jugadores registrados en un DataGridView
- Uso de Programación Orientada a Objetos (POO)

Se aplicaron conceptos fundamentales de POO mediante:

- Clases
- Objetos
- Encapsulamiento
- Métodos
- Colecciones (List)

Esto permitió una mejor organización y mantenimiento del código.

Almacenamiento de información

La información de los jugadores se maneja mediante listas en memoria y almacenamiento en archivo .txt, evitando el uso de bases de datos y permitiendo conservar los registros aun después de cerrar la aplicación.

Uso de eventos

Se emplearon eventos como Click en los botones y selección de filas en el DataGridView para ejecutar acciones específicas como registrar, modificar, eliminar y mostrar información automáticamente.

Control	Propiedad	Valor
Form1	Text	Sistema de Alineación de Futbol
Form1	BackColor	WhiteSmoke
lblTitulo	Text	ALINEACIÓN FUTBOL
lblTitulo	ForeColor	DarkBlue
txtNombre	Name	txtNombre
txtNombre	Size	250, 30
cboPosicion	Items	Arquero, Defensa, Mediocampista, Delantero
cboCondicion	Items	Titular, Suplente
nudNumero	Minimum	1
nudNumero	Maximum	99
btnAgregar	BackColor	SeaGreen
btnAgregar	Text	AGREGAR
btnModificar	BackColor	DarkOrange
btnModificar	Text	MODIFICAR
btnEliminar	BackColor	Firebrick
btnEliminar	Text	ELIMINAR
dgvJugadores	ReadOnly	True
dgvJugadores	SelectionMode	FullRowSelect
lblFecha	Text	Fecha del sistema

3. Aplicaciones y casos de uso:

Sistema de Alineación de Futbol

ALINEACIÓN FUTBOL

Datos del Jugador

Nombre Jugador

Posición

Número Camiseta

Condición

AGREGAR **MODIFICAR** **ELIMINAR**

Fecha del sistema

```

1 namespace Alineación_de_Futbol
2 {
3     4 referencias
4     public class Jugador
5     {
6         // ENCAPSULAMIENTO
7         private string nombre;
8         private string posicion;
9         private int numero;
10        private string condicion;
11
12        // PROPIEDADES
13        4 referencias
14        public string Nombre
15        {
16            get { return nombre; }
17            set { nombre = value; }
18        }
19
20        4 referencias
21        public string Posicion
22        {
23            get { return posicion; }
24            set { posicion = value; }
25        }
26
27        3 referencias
28        public int Numero
29        {
30            get { return numero; }
31            set { numero = value; }
32        }
33    }
34 }

```

```

public string Condicion
{
    get { return condicion; }
    set { condicion = value; }
}

// MÉTODO
public string MostrarJugador()
{
    return Nombre + " - " + Posicion;
}
}

```

```

namespace Alineación_de_Futbol
{
    3 referencias
    public partial class Form1 : Form
    {
        // COLECCIÓN LIST
        List<Jugador> listaJugadores = new List<Jugador>();

        // VARIABLE PARA SABER LA FILA SELECCIONADA
        int indiceSeleccionado = -1;

        1 referencia
        public Form1()
        {
            InitializeComponent();
        }

        // CARGA DEL FORMULARIO
        1 referencia
        private void Form1_Load(object sender, EventArgs e)
        {
            lblFecha.Text = "Fecha Actual: " + DateTime.Now.ToString();

            MostrarJugadores();
        }
    }
}

```



```

3 // MÉTODO PARA MOSTRAR JUGADORES
4 4 referencias
5 private void MostrarJugadores()
6 {
7     dgvJugadores.DataSource = null;
8     dgvJugadores.DataSource = listaJugadores;
9 }
10
11 // MÉTODO LIMPIAR
12 3 referencias
13 private void LimpiarCampos()
14 {
15     txtNombre.Clear();
16
17     cboPosicion.SelectedIndex = -1;
18     cboCondicion.SelectedIndex = -1;
19
20     nudNumero.Value = 1;
21
22     txtNombre.Focus();
23 }

```

```

4 4 referencias
5 private void btnAgregar_Click(object sender, EventArgs e)
6 {
7     // VALIDACIONES
8     if (txtNombre.Text.Trim() == "")
9     {
10         MessageBox.Show("Ingrese el nombre del jugador");
11         return;
12     }
13
14     if (cboPosicion.SelectedIndex == -1)
15     {
16         MessageBox.Show("Seleccione la posición");
17         return;
18     }
19
20     if (cboCondicion.SelectedIndex == -1)
21     {
22         MessageBox.Show("Seleccione la condición");
23         return;
24     }
25
26     // CREACIÓN DEL OBJETO
27     Jugador jugador = new Jugador();
28
29     jugador.Nombre = txtNombre.Text;
30     jugador.Posicion = cboPosicion.Text;
31     jugador.Numero = Convert.ToInt32(nudNumero.Value);
32     jugador.Condicion = cboCondicion.Text;
33
34     // AGREGAR A LA LISTA
35     listaJugadores.Add(jugador);
36
37     // MOSTRAR DATOS
38     MostrarJugadores();
39
40     MessageBox.Show("Jugador agregado correctamente");
41
42     LimpiarCampos();
43 }

```

```
// MODIFICAR JUGADOR
1 referencia
private void btnModificar_Click(object sender, EventArgs e)
{
    if (indiceSeleccionado == -1)
    {
        MessageBox.Show("Seleccione un jugador");
        return;
    }

    listaJugadores[indiceSeleccionado].Nombre = txtNombre.Text;
    listaJugadores[indiceSeleccionado].Posicion = cboPosicion.Text;
    listaJugadores[indiceSeleccionado].Numero = Convert.ToInt32(nudNumero.Value);
    listaJugadores[indiceSeleccionado].Condicion = cboCondicion.Text;

    MostrarJugadores();

    MessageBox.Show("Jugador modificado correctamente");

    LimpiarCampos();

    indiceSeleccionado = -1;
}
```

```
// ELIMINAR JUGADOR
1 referencia
private void btnEliminar_Click(object sender, EventArgs e)
{
    if (indiceSeleccionado == -1)
    {
        MessageBox.Show("Seleccione un jugador");
        return;
    }

    DialogResult respuesta;

    respuesta = MessageBox.Show(
        "¿Desea eliminar este jugador?",
        "Confirmación",
        MessageBoxButtons.YesNo,
        MessageBoxIcon.Question);

    if (respuesta == DialogResult.Yes)
    {
        listaJugadores.RemoveAt(indiceSeleccionado);

        MostrarJugadores();

        MessageBox.Show("Jugador eliminado correctamente");

        LimpiarCampos();

        indiceSeleccionado = -1;
    }
}
```

```
// SELECCIONAR FILA DEL DATAGRIDVIEW
1 referencia
private void dgvJugadores_CellContentClick(object sender, DataGridViewCellEventArgs e)
{
    if (e.RowIndex >= 0)
    {
        indiceSeleccionado = e.RowIndex;

        txtNombre.Text =
            listaJugadores[indiceSeleccionado].Nombre;

        cboPosicion.Text =
            listaJugadores[indiceSeleccionado].Posicion;

        nudNumero.Value =
            listaJugadores[indiceSeleccionado].Numero;

        cboCondicion.Text =
            listaJugadores[indiceSeleccionado].Condicion;
    }
}
}
```

CONCLUSIÓN

Se aplicaron conceptos fundamentales de Programación Orientada a Objetos (POO), tales como clases, objetos, encapsulamiento, métodos y colecciones (List), permitiendo administrar la información de los jugadores de manera organizada y eficiente.

Asimismo, se utilizaron controles gráficos, manejo de eventos, validaciones de datos y operaciones CRUD (Agregar, Modificar, Eliminar y Mostrar), logrando una interfaz amigable y fácil de utilizar para el usuario.

En conclusión, este laboratorio permitió fortalecer habilidades en el desarrollo de aplicaciones de escritorio, programación orientada a objetos y diseño de interfaces gráficas, demostrando la utilidad de estas herramientas en la creación de soluciones prácticas para la gestión de información.

REFERENCIAS BIBLIOGRÁFICAS

Microsoft. (2024). Introducción a Windows Forms. Microsoft Learn.

<https://learn.microsoft.com/es-es/dotnet/desktop/winforms/>

Microsoft. (2024). Eventos en C#. Microsoft Learn.

<https://learn.microsoft.com/es-es/dotnet/csharp/programming-guide/events/>

Microsoft. (2024). Controles en Windows Forms. Microsoft Learn.

<https://learn.microsoft.com/es-es/dotnet/desktop/winforms/controls/>